CRITICAL SECURITY & ETHICS WARNING

Read Before Proceeding with the Lab

You are about to handle live, dangerous malware. This is not a simulation. The samples we are using are real, malicious, and designed to cause catastrophic damage, including destroying your data, spreading across networks, and stealing personal information.

Failure to follow these instructions precisely can result in the infection of your personal computer, the college/university network, and the computers of others. You are personally responsible for any damage caused by negligence.

Security Rules (Non-Negotiable)

1. **ABSOLUTE ISOLATION:** Your analysis Virtual Machine (VM) must be completely isolated at all times.
    a. **Network Setting:** The VM's network adapter must be set to "Host-Only Network" or an "Internal Network." It must never be set to "Bridged," "NAT," or "NAT Network."
    b. **Verification:** Before you begin, you *must* prove your VM is isolated. Try to ping google.com from inside your Windows VM. The request must fail. If it succeeds, STOP IMMEDIATELY and fix your network settings.
2. **NO HOST INTERACTION:** Your host operating system (your main Windows, macOS, or Linux desktop) is your "clean room."
    a. Do NOT download the malware samples directly to your host machine.
    b. Do NOT copy *any* personal files, documents, or schoolwork *into* the analysis VM.
    c. Do NOT copy *any* files from the analysis VM back to your host. All reports must be written separately.
3. **SNAPSHOTS ARE YOUR LIFELINE:** We are using snapshots to ensure your safety.
    a. You must take a "Clean" snapshot of your Windows VM *before* you introduce the malware.
    b. After *every* analysis session where you run the malware, you must shut down the VM and revert to the clean snapshot. This is the *only* way to guarantee the malware is gone. Do not "save" the infected state.
4. **DO NOT SHARE SAMPLES:** The malware samples must stay within this lab environment. Do **not** email them, upload them to Dropbox/Google Drive, or send them to anyone.

Legal & Ethical Disclaimer

- The techniques and tools used in this lab are for educational and defensive purposes only.
- The skills you are learning (reverse engineering, malware analysis) are the same ones used by cybersecurity professionals to defend networks and by malicious actors to attack them.
- Using these skills to analyze any software or system without explicit, written permission is illegal and a serious crime under laws such as the Computer Fraud and Abuse Act (CFAA) and others.
- By participating in this lab, you agree to use this knowledge ethically and legally. You are solely responsible for your actions.

LAB17: MALWARE ANALYSIS PART 1

Primary Objective: To set up a secure, isolated "clean room" environment, safely acquire the malware sample, and perform initial static analysis using Kali Linux to form a hypothesis about its function.

Here is the consolidated, step-by-step guideline for Day 1, incorporating the direct download workflow and the detailed pe-bear build.

Critical Security Rules (Reminder):

- This lab involves live, dangerous malware.
- Isolation is not optional. Once the malware is on your VM, it must never have an internet connection again.

Part 1: Acquisition (The "Online" Phase)

1. **Set Kali to NAT Mode:**
    - Go to your Kali VM's Settings > Network.
    - Set "Attached to:" to "NAT". (This gives it internet access).

Part 2: Build pe-bear from Source (on Kali)

1. Start your Kali VM (it should still be running).
2. Update Package Lists:

```bash
sudo apt update
```

1. **Install Build Dependencies:** This command gets all the tools and libraries needed to compile the program.

```bash
sudo apt install git build-essential cmake qt5-qmake qtbase5-dev
```

Press Y to approve the installation.

2. **Download the Source Code (Git Clone):**

```bash
git clone --recursive
https://github.com/hasherezade/pe-bear.git
```

3. **Run the Build Script:**

```bash
./build.sh
```

This will take a few minutes. You should see it [Build] up to [100%] and mention the output path.

Part 3: Download Sample

- Open Firefox, navigate *directly* to your malware source (e.g., theZoo on GitHub), and download the password-protected WannaCry ZIP file (password: infected). Move the file from downloads to the you created.
- Once the file has been downloaded successfully, change the network settings from NAT to Host-only.

Part 4: Static Triage (on Kali)

Extract the Malware:

- Navigate to the folder where you copied the downloaded the file
- Unzip the file. Make sure you are outside the pe-bear folder:

```bash
unzip -P infected [name_of_zip_file].zip
```

The unzipped file will have a weird name. Rename it to wannacry.exe for ease. The following snapshot shows how your terminal might look if you have successfully completed all the steps so far.

Rename the file using the mv command to wanncry.exe.

```
┌──(kali㉿kali)-[~/Desktop/lab17]
└─$ unzip -P infected Ransomware.WannaCry.zip
Archive:  Ransomware.WannaCry.zip
  inflating: ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa.exe

┌──(kali㉿kali)-[~/Desktop/lab17]
└─$ ls
ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa.exe  pe-bear  Ransomware.WannaCry.zip

┌──(kali㉿kali)-[~/Desktop/lab17]
└─$ mv ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa.exe wannacry.exe

┌──(kali㉿kali)-[~/Desktop/lab17]
└─$ ls
pe-bear  Ransomware.WannaCry.zip  wannacry.exe
```

Run file Command:

```bash
file wannacry.exe
```

- Purpose: This tells you what kind of file it is.
- Expected Output: wannacry.exe: PE32 executable (GUI) Intel 80386, for MS Windows.
- Analysis: This confirms it's a 32-bit Windows program with a Graphical User Interface (GUI), not just a command-line tool.

```
┌──(kali㉿kali)-[~/Desktop/lab17]
└─$ file wannacry.exe
wannacry.exe: PE32 executable for MS Windows 4.00 (GUI), Intel i386, 4 sections
```

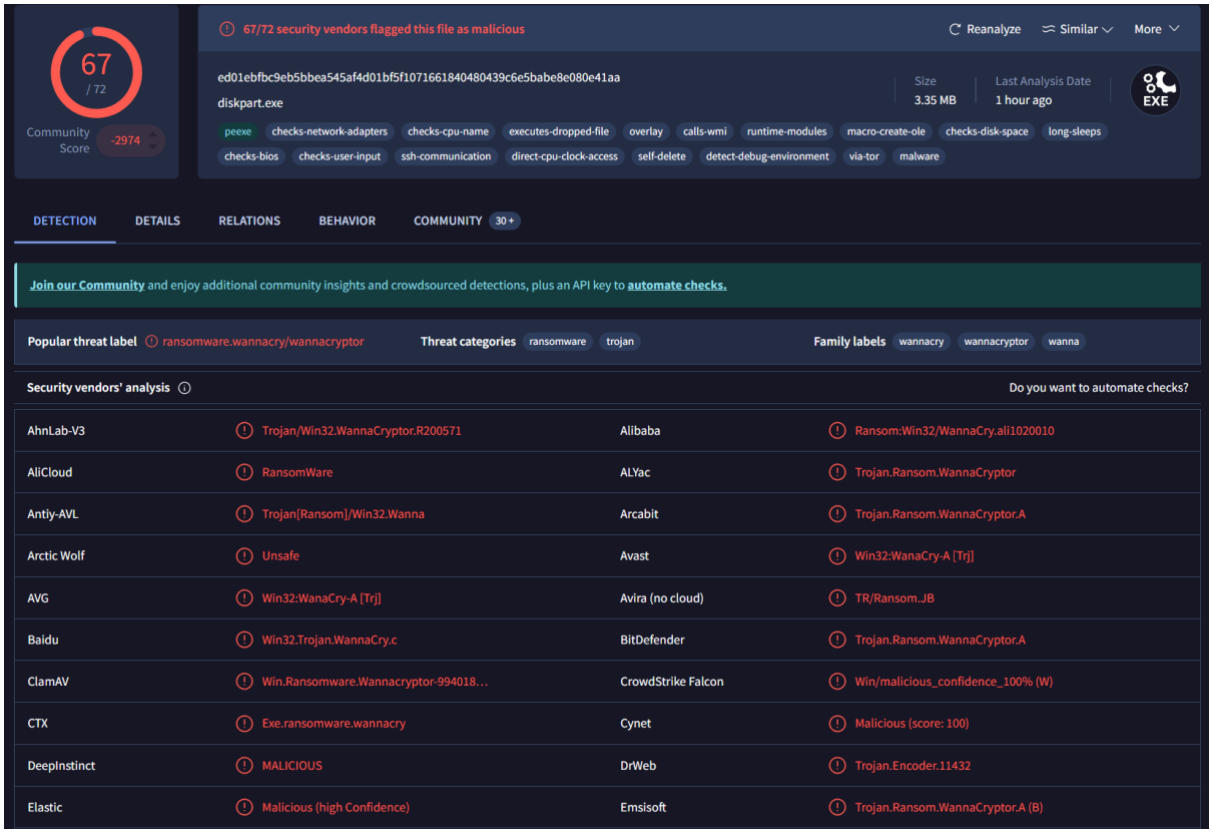
Run sha256sum Command (Hashing):

- Command: sha256sum wannacry.exe
- Purpose: This generates a unique digital fingerprint (a "hash") for the file.
- Expected Output: A long string of characters, like db349b97c37d22f5ea1d180b0c03f41e544...

```
┌──(kali㉿kali)-[~/Desktop/lab17]
└─$ sha256sum wannacry.exe
ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa  wannacry.exe
```

Check Hash on VirusTotal (on Host):

- Copy this hash.
- Go back to your Host machine's browser.
- Go to virustotal.com.
- Paste the hash into the "Search" bar.
- Analysis: Review the report. You will see its common name ("WannaCry"), its high detection rate, and other aliases. This confirms what you are dealing with.



Part 5: PE Structure Analysis (on Kali)

1. **Run your newly built pe-bear:**
   - First, cd back to the pe-bear directory: cd ~/pe-bear (or wherever you cloned it).

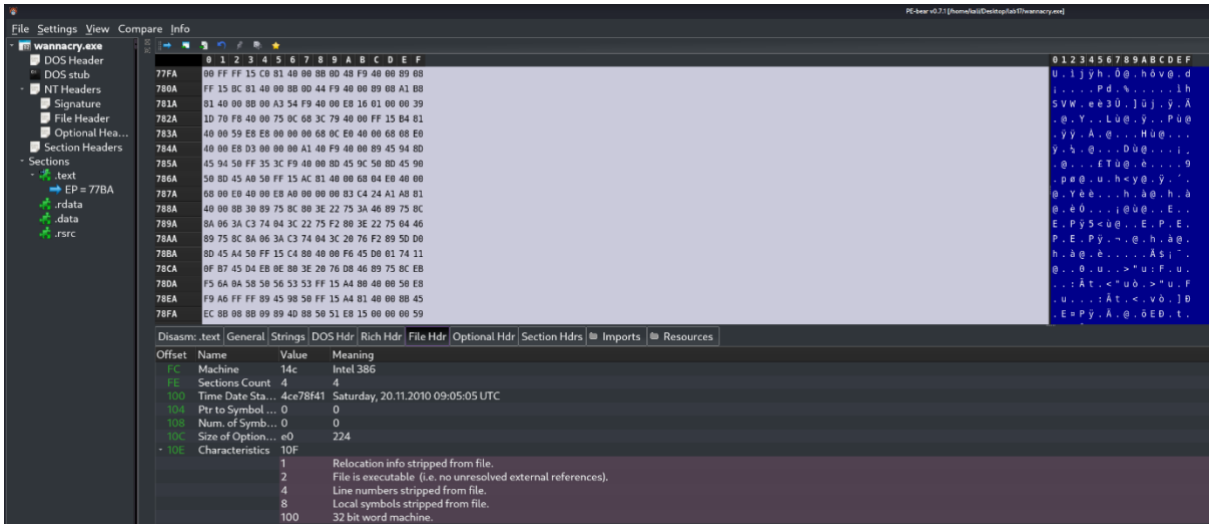
- Run the executable:

```
LAB 17                                Bash
./build_qt6/bin/PE-bear
```

Analyze the "File Hdr" Tab:

1. pe-bear opens on a summary screen. Look for these fields:
2. Time Date Stamp: You'll see a timestamp, like 2017-05-12. This is the *compile time*. This (or a similar date) is when the malware authors compiled this specific version. (Note: this can be faked, a technique called "timestomping").

3. Machine: It will say I386 (32 bit). This confirms our 32-bit analysis path.



Analyze the "Section Hdr" (Section Headers):

1. You'll see a list: .text, .rdata, .data, .rsrc.
2. .text: This is the *code*. Note its "Permissions." It should be R (Read) and X (Execute).
3. .rdata: This is the *read-only data*. The strings you found are likely here. Click on it. The hex view below will jump to that section. You might even see the file extension list or the WanaCrypt0r string in the data.
4. .rsrc: This is for *resources*. This is where things like icons, images, and (in our case) the ransom note text and language translations are stored.

Name	Raw Addr.	Raw size	Virtual Addr.	Virtual Size	Characteristics	Ptr to Reloc.	Num. of Reloc.	Num. of Linenum.
.text	1000	7000	1000	69B0	60000020	0	0	0
>	8000	^	8000	mapped: 7000	r-x			
.rdata	8000	6000	8000	5F70	40000040	0	0	0
>	E000	^	E000	mapped: 6000	r--			
.data	E000	2000	E000	1958	C0000040	0	0	0
>	10000	^	10000	mapped: 2000	rw-			
.rsrc	10000	34A000	10000	349FA0	40000040	0	0	0
>	35A000	^	35A000	mapped: 34A000	r--			

Analyze the "Imports" (The Most Important Part):

1. This is the *definitive list* of Windows functions the malware plans to use. It's far more reliable than strings.
2. Have your students look for the DLLs and the functions that confirm our hypotheses. They will find:
    a. **KERNEL32.DLL:**
        - CreateFileA / WriteFile (File I/O - confirms it reads/writes files)
        - CreateProcessA (Spawning new commands - like the taskkill we saw)
        - CreateServiceA / StartServiceA (Confirms our "persistence" hypothesis)
        - CreateMutexA (Confirms our MsWinZonesCacheCounterMutexA hypothesis)
    b. **ADVAPI32.DLL:**
        - CryptGenKey
        - CryptEncrypt
        - CryptDestroyKey
        - (This is the smoking gun for encryption). This *proves* it's not just *called* "WanaCrypt0r," it *actually performs* cryptography.

TASK

WININET.DLL is a Windows library that contains functions for networking. Specifically, it includes functions like InternetOpenA, which is used to initialize a network connection, and InternetOpenUrlA, which is used to connect to a specific URL, in this case, the malware's kill switch URL.

Find WININET.DLL in the **'Imports'** header and look for:

- InternetOpenA (Network connection)
- InternetOpenUrlA (Connecting to the kill switch URL)

WININET.DLL is not in this list. This tells us something *very* important: the malware is intentionally hiding its networking capabilities.

This is a common "defense evasion" technique. Instead of listing WININET.DLL in the header (which is like announcing "I make network connections!" to analysts), the malware "finds" and "loads" the networking functions *manually* after it has already started running.

The New "Smoking Gun"

Here is your new task. This will confirm our hypothesis.

1. In pe-bear, click on the KERNEL32.DLL entry to see the 54 functions it imports.
2. Look for these two functions in that list:
   - LoadLibraryA
   - GetProcAddress

Finding these two functions is the "smoking gun" for this technique.

- LoadLibraryA is the function a program uses to load any DLL it wants *at runtime*.
- GetProcAddress is what it uses to find the *address* of a specific function (like InternetOpenUrlA) inside that DLL.

So, the malware's logic (which we'll see in Ghidra) will be:

1. Call LoadLibraryA with the string "w-i-n-i-n-e-t-.-d-l-l".
2. Call GetProcAddress to find InternetOpenUrlA.
3. Call that function to connect to the kill switch.

Before we finish Day 1, let's confirm the *other* imports:

- Click on ADVAPI32.DLL. Do you see CryptGenKey and CryptEncrypt?
- Click on KERNEL32.DLL. Do you see CreateServiceA and CreateMutexA?

KERNEL32.DLL Analysis:

- LoadLibraryA: This is the "master key." It lets the malware load *any other DLL* on the system that it *didn't* want to list in its header.
- GetProcAddress: This is the "lockpick." After it loads a hidden DLL, it uses this function to find any secret function it wants, like InternetOpenUrlA (for networking) or CryptEncrypt (for encryption).
- CreateMutexA: You also found this. This confirms our mutex hypothesis from the strings output.

Conclusion: We now **know** *how* it hides its networking. It's not listed because it loads WININET.DLL manually using LoadLibraryA.

ADVAPI32.DLL Analysis

- CreateServiceA
- StartServiceA
- OpenSCManagerA

Conclusion: This 100% confirms the persistence mechanism. We now know it creates a Windows service to survive reboots.

Big Mystery

You're probably thinking, "Wait, just like WININET.DLL, I don't see CryptGenKey or CryptEncrypt in this list either!"

You are exactly right. And the answer is the same.

The malware authors were *so* paranoid about analysts seeing their encryption functions that they hid those too. They use the *exact same* LoadLibraryA/GetProcAddress trick to manually load the crypto functions they need from ADVAPI32.DLL.

STRING ANALYSIS

Disasm: .rdata | General | Strings | DOS Hdr | Rich Hdr | File Hdr | Optional Hdr | Section Hdrs | Imports | Resources

Extracted strings: 17021

Save | Page 0 | Max per page 10000 | Search string Mutex

	Offset	Type	Length	
177	da86	A	10	OpenMutexA
326	f4b4	A	35	Global\MsWinZonesCacheCounterMutexA

Disasm: .rdata | General | Strings | DOS Hdr | Rich Hdr | File Hdr | Optional Hdr | Section Hdrs | Imports | Resources

Extracted strings: 17021

Save | Page 0 | Max per page 10000 | Search string Crypt

	Offset	Type	Length	
201	dc16	A	19	CryptReleaseContext
262	e034	W	11	WanaCrypt0r
313	f08c	A	53	Microsoft Enhanced RSA and AES Cryptographic Provider
314	f0c4	A	11	CryptGenKey
315	f0d0	A	12	CryptDecrypt
316	f0e0	A	12	CryptEncrypt
317	f0f0	A	15	CryptDestroyKey
318	f100	A	14	CryptImportKey
319	f110	A	20	CryptAcquireContextA

1. Mutex Findings (The "Is anyone else here?" check)

Our first screenshot confirms our hypothesis about the mutex:

- Global\MsWinZonesCacheCounterMutexA: This is the name of the mutex. Think of it as a unique "calling card" the malware leaves in the system. The Global\ prefix means it's visible across all user sessions.
- OpenMutexA: This is the action. This is the Windows function the malware calls at startup to try and "open" a mutex with that specific name.

Conclusion: This proves the malware performs a self-check. When it first runs, it calls OpenMutexA to look for Global\MsWinZonesCacheCounterMutexA. If it finds it (meaning OpenMutexA succeeds), it

knows another copy is already running and it will immediately exit. This is a common technique to prevent the malware from infecting a machine multiple times.

2. Encryption Findings (The "Smoking Gun")

This second screenshot is the **jackpot**. It confirms every suspicion about the ransomware payload.

- WanaCrypt0r: This is the malware's self-proclaimed name.
- CryptEncrypt, CryptGenKey, CryptDestroyKey, etc...: This is the malware's entire toolkit. We've found the names of all the Windows functions it will use to perform encryption.
- Microsoft Enhanced RSA and AES Cryptographic Provider: This is the most critical string. It tells us the exact type of encryption it uses: a hybrid RSA (asymmetric) and AES (symmetric) scheme. This is a classic, strong encryption method used by almost all modern ransomware.

Conclusion: We have *proven* this file is ransomware, what it's called, and the exact cryptographic library and functions it will use to encrypt files.

The Big Mystery (That We've Set Up for Part 2)

Here is the final, brilliant connection:

1. In the Strings tab: We just found all the Crypt... function *names*.
2. In the Imports tab (earlier): We confirmed those functions were not listed.

This proves our theory from before: the malware is intentionally hiding its capabilities. It uses LoadLibraryA and GetProcAddress to secretly load these crypto functions at runtime, all to make our job (static analysis) harder.

This discovery—that the malware is *hiding* its imports—is the perfect conclusion to Lab17 Part 1. We've pushed static file-header analysis to its limit. The only way to find out *how* it uses LoadLibraryA is to look at the actual code.

GLOSSARY

Windows Executable (PE) Structure

This is the "blueprint" for a Windows .exe or .dll file. It tells the operating system how to load the program into memory and run it.

- **PE32 (Portable Executable):** This is the standard file format for all 32-bit Windows applications. The "Portable" part means the format is designed to be recognized by all versions of Windows. Your lab's wannacry.exe is a PE32 file, confirming it's a 32-bit program.
- **Section Headers:** Think of this as the file's "Table of Contents". Each header points to a different section of the file (like .text or .rsrc), describes what's in it (code, data, etc.), states its size, and (most importantly) sets its permissions (e.g., "Read & Execute" or "Read & Write").
- **.text Section**: This is where the program's actual machine code instructions live. This is the part of the file the CPU reads and executes. For security, this section is almost always marked as Read-Only and Execute-Only. It should *not* be writable; if malware tries to make it writable, it's a huge red flag that it's trying to modify its own code in memory (a technique called "self-modifying code").
- **.data Section:** This section holds global data—variables that can be accessed and *changed* by any part of the program. For example, a counter or a user-input string would be stored here. It is typically marked as Read/Write.
- **.rdata Section:** This holds read-only data. This is for information that the program needs to read but should *never* change, like configuration strings, the "WanaCrypt0r" string, or the list of imported functions (the Import Table).
- **.rsrc (Resource Section):** This section is a "storage closet" for all the program's non-code, non-data assets. This includes icons (what you see in File Explorer), images (like a company logo), menu layouts, and, in WannaCry's case, the ransom note text itself, often in multiple languages.
- **Timestomping:** This is an anti-forensics technique. Every file has timestamps (Created, Modified, Accessed). Malware authors will deliberately change these timestamps to make the file look like a legitimate, old system file (e.g., dated 2017 to match the original WannaCry). This is done to blend in, avoid suspicion, and confuse forensic analysts who are trying to build a timeline of the attack.

Malware Techniques & Windows Functions

These are the "verbs" (functions) and "tools" (DLLs) the malware uses to perform its malicious actions.

- **DLL (Dynamic Link Library):** A DLL is a shared toolbox. Instead of every program having its *own* copy of "how to open a file," Windows provides a central DLL (like KERNEL32.DLL) with a CreateFile function that any program can use. Malware abuses these legitimate "tools" for malicious purposes.
- **Imports (Import Table):** This is the "official list" in the PE file's .rdata section that tells Windows exactly which functions the program needs from which DLLs. It's like a shopping list. An analyst can look at this list and see "Ah, it imports CreateServiceA (persistence) and CryptEncrypt (encryption). This is definitely malware."
- **Hiding Imports (Runtime Loading):** This is the primary defense evasion technique from your lab. Because the Import Table is so revealing, paranoid malware authors leave it *blank*. Instead of "officially" importing functions, the malware secretly finds and loads them *after* it's already running. This is done with two key functions:
    - LoadLibraryA: This function's job is to load any DLL from disk into the program's memory at runtime.

    - GetProcAddress: Once the DLL is loaded, this function "searches" it for a specific function by name (like "InternetOpenUrlA") and returns its memory address. The malware then calls that address directly, completely bypassing the Import Table and blinding static analysis tools.
- **KERNEL32.DLL:** The "heart" of Windows. It manages the most basic, fundamental operations: memory management, file input/output (I/O), and creating/managing processes and threads. It's essential for any program to run.
- **ADVAPI32.DLL:** The "Advanced API" library. It handles higher-level functions, most notably security, managing the Windows Registry, and (critically for malware) creating, starting, and stopping Windows Services.
- **WININET.DLL:** The "Internet" DLL. This library provides easy-to-use functions for communicating over the internet using protocols like HTTP and FTP. Malware uses this to download more payloads, send stolen data to an attacker, or, in WannaCry's case, connect to its kill switch URL.
- **Cryptography (Hybrid Encryption):** This is the "payload." WannaCry uses a strong, two-layer approach called hybrid encryption:
    - **AES (Symmetric):** A very fast, strong algorithm. The malware generates a *new* AES key for each victim. It uses this key to do the "bulk encryption" of all the user's files.
    - **RSA (Asymmetric):** A slower but incredibly secure algorithm. The malware has the attacker's public key embedded inside it. After encrypting the files with AES, it encrypts the AES key itself using the RSA public key. The victim can't decrypt their files without the AES key, and they can't get the AES key because it's locked by RSA. Only the attacker, who has the matching private RSA key, can unlock it.
- **CryptGenKey / CryptEncrypt:** These are the specific Windows functions (from ADVAPI32.DLL) that do the work. CryptGenKey is called to generate a cryptographic key, and CryptEncrypt is the function that uses that key to perform the actual encryption on data.
- **Kill Switch:** A "self-destruct" mechanism. The WannaCry malware would try to connect to a specific, random-looking domain. If the connection *succeeded*, the malware would assume it was being analyzed in a sandbox (which often fake-reply to all network requests) and would stop running, exiting immediately. A security researcher accidentally activated this, which helped stop the malware's spread.
- **Mutex (Global\\MsWinZonesCacheCounterMutexA):** A mutex is a "token" used to prevent two processes from doing the same thing at the same time. Malware uses a named mutex as a "check-in" system. When WannaCry first runs, it tries to create the mutex Global\\MsWinZonesCacheCounterMutexA. If it succeeds, it knows it's the first copy and continues to infect the machine. If it *fails* (because the mutex already exists), it knows another copy is already running, so it just exits immediately. This prevents it from re-infecting the same machine over and over.
- **Persistence (Windows Service):** This is any technique malware uses to survive a reboot. WannaCry uses one of the most powerful methods: creating a new Windows Service. A service is a program that runs in the background with high privileges (often as the SYSTEM user) and is set to start automatically when the computer boots up—even before a user logs in. The malware uses these functions (found in ADVAPI32.DLL) to install its service:
    - CreateServiceA: This function registers the malware as a new service with the system.
    - StartServiceA: This function immediately starts the newly created service.